---------/Part 1 Completed/----------

Part 2 (OOPS)

1. Object and class
2. Members of Class
3. Initializing Properties of object.
4. Constructor overloading
5. Constructor Chaining vscode
6. Access specifier & Package eclipse
7. Playing with objects
8. Encapsulation
9. Relationship
10. Inheritance
11. Non Primitive type Casting.
12. Polymorphism
    - Method overloading
    - Method overriding.
13. Abstraction.

Day46

Date: 13/10/2024
Day: Tuesday

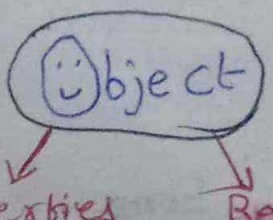
CHAPTER - 1

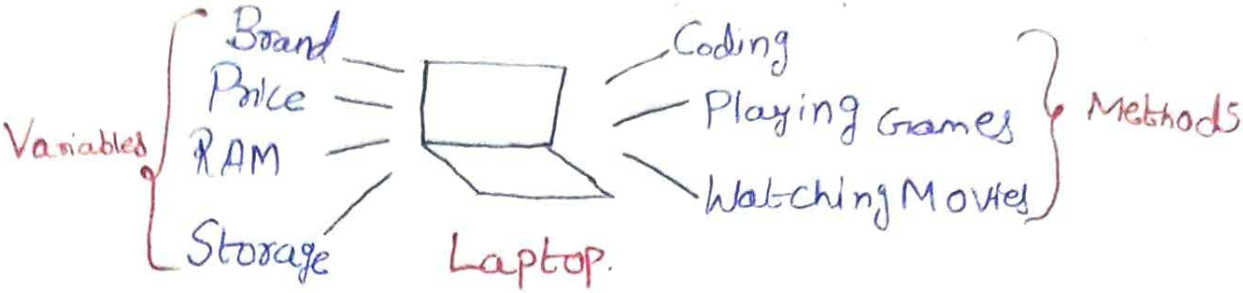
1. Object & Class.

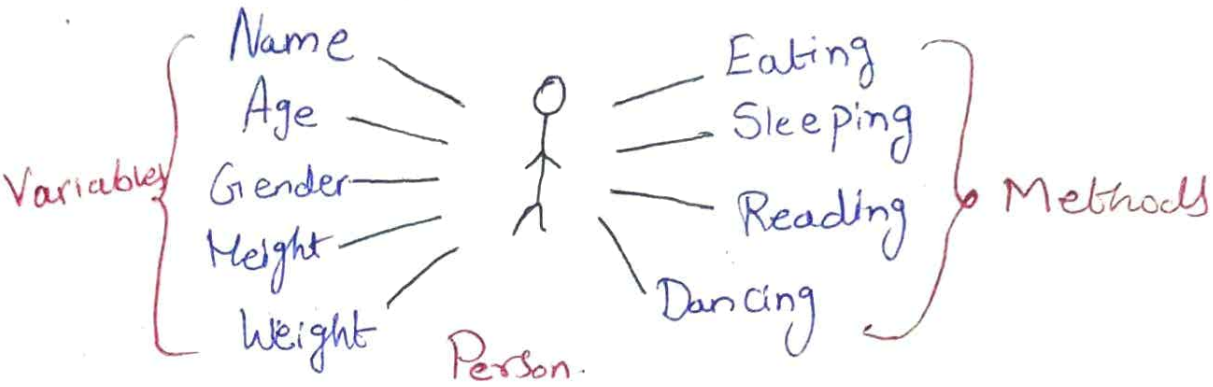
☺bject:- Anything which is present in real World, and Having Physical Existance is known as ☺bject.

Ex- Car, Bike, Bag, Book, Pawan Sir etc.

There are 'N' no of objects are present in this world. Every object are having their own Properties & Behaviour.



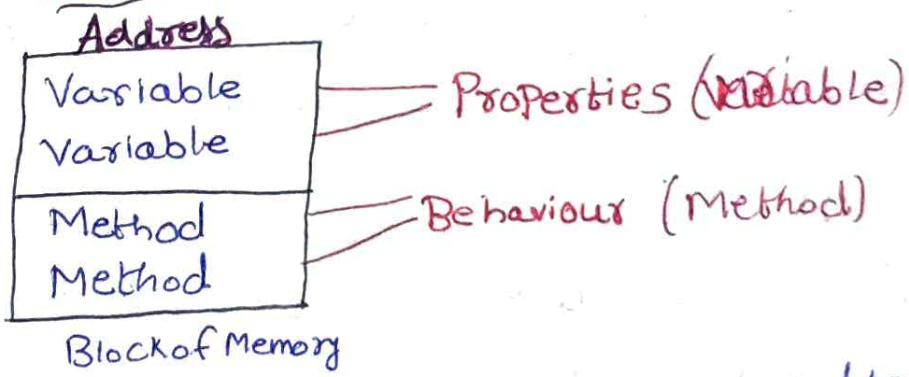




All the Properties of Objects representing by using Variables

All the Behaviour of objects representing by using Methods.

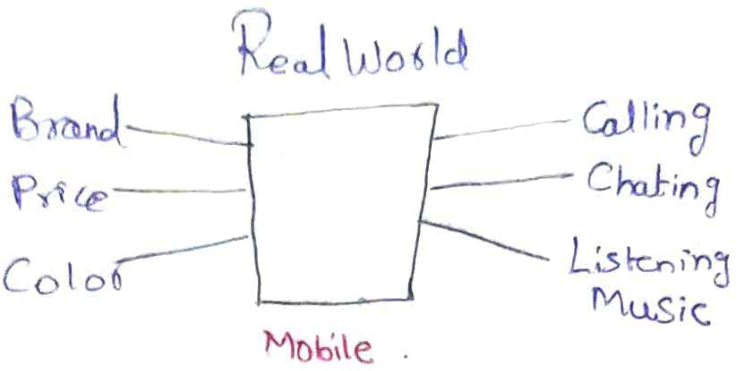
Objects in Java.

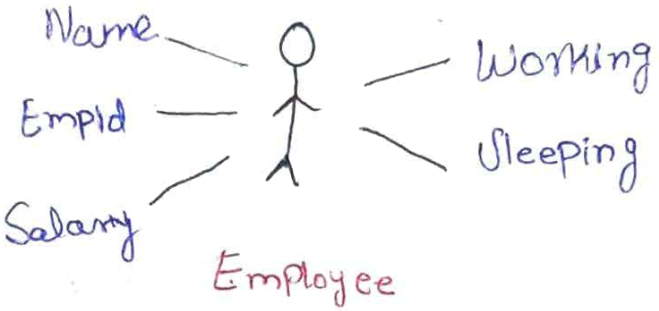


Object is a Address Block of Memory, which is used to Store Multiple Variables and Multiple Methods

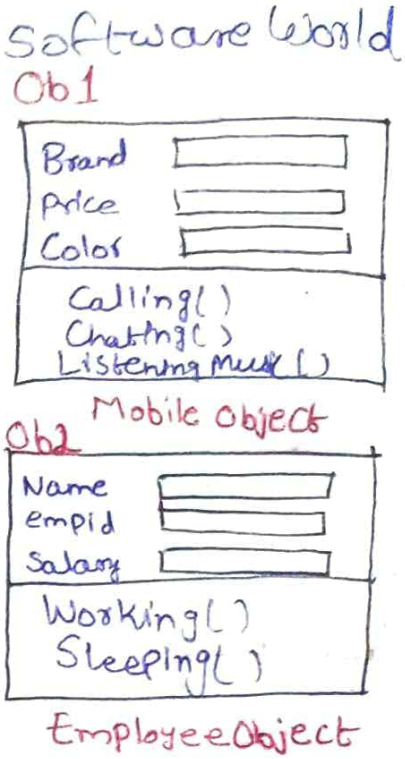
Why we need Object in Java?

To Represent Real World inside Software world.
We should Create Object.





Software World.



How to Create Object

There are 2 steps as to be followed to create an object

Step 1: Creating the BluePrint of an Object.

↳ Class is used to Create blue print of an object.

Class: Class is a Keyword, which is used to Create Class Block & Class is also act as an BluePrint of an object.

```
Class ClassName
{
    // Properties[V]
    // Behaviour [M]
}
```

CB

Step 2: Syntax.

```
new ClassName();
```

new → Keyword

ClassName() → Constructor Calling Statement.

Day 47.

Date: 14/10/2026
Day: Wednesday.

Mock Test – Part - 1

Day 48.

Date: 15/10/2026
Day: Thursday

Mock Test Part - 1

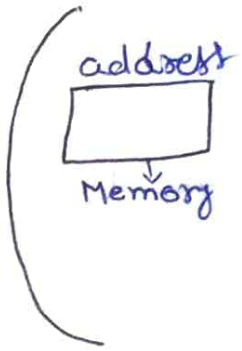
Day 49

Date: 16/10/2026
Day: Friday.

new: new is a keyword which is used to Create an object inside a Heap area.

→ Every object are having Unique Address
→ At the Last new keyword is returning the address

```
new ClassName();
 ↓
address
```

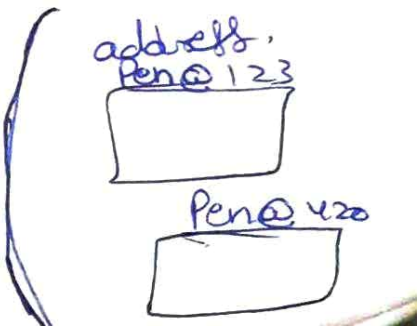


How to create an empty object/area.

```
Class Pen {
 // Properties [V]
 // Behaviour [M]
 p. s. v. m (String [] args) {
        S.o.Pln( new Pen());
        S.o.pln(new Pen());
    }
}
```

Process

O/P: Pen@123
Pen@420.



Note:- Whenever we are creating a class for Blueprint Purpose, we should not use that class for execution Purpose

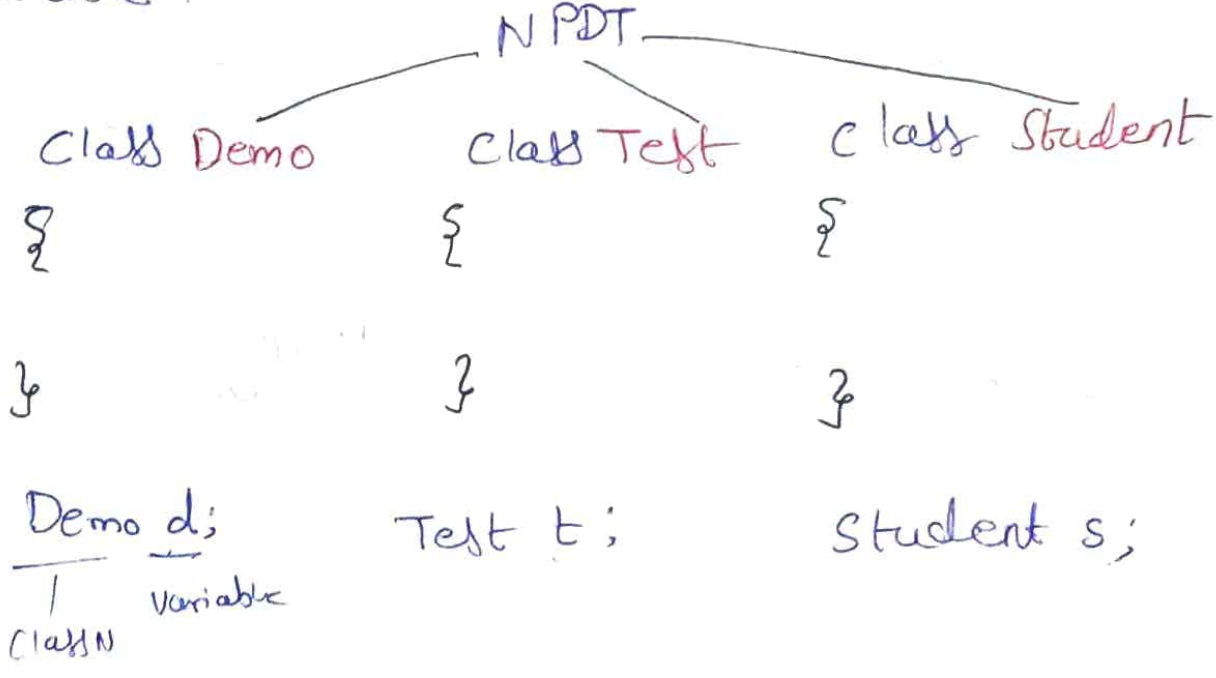
→ For execution Purpose, we should create a Separate class called Driver Class / Main Class

→ To store the address of the object, we should create Object reference variable.

Object Reference variable [NPV].

A variable which is used to store the object address is known as Object Reference variable.

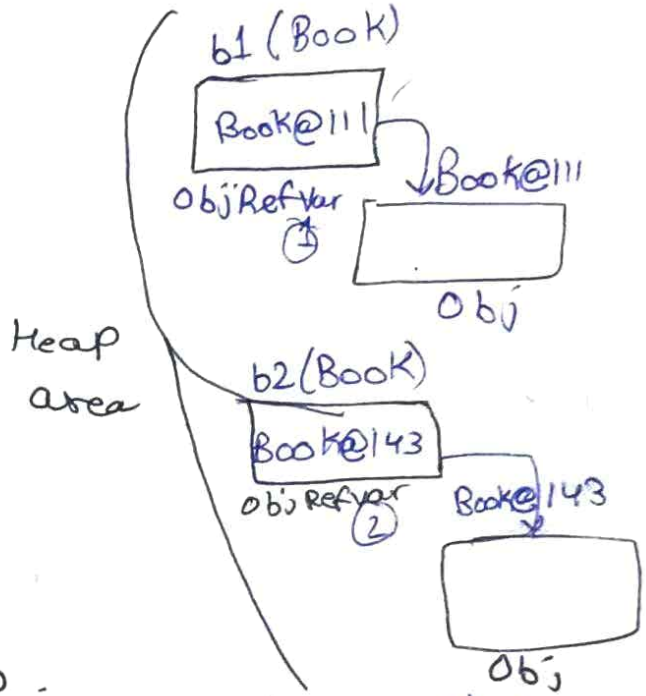
NPDT is used to create the object Reference variable.



```
class Book {

}

class BookDriver {
    Public Static void main (String [] args) {
        Book b1 = new Book ();
        Book b2 = new Book ();
        b2 = new
        S.o.pln (b1);
        S.o.pln (b2);
        S.o.pln (b1 == b2); // False .
    }
}
```



Day 50.

Date :- 17/10/2026

No class

Day :- Saturday

Day 51

Date :- 21/10/2026

Day :- Wednesday

No Class

Date :- 22/10/2026

Day 52

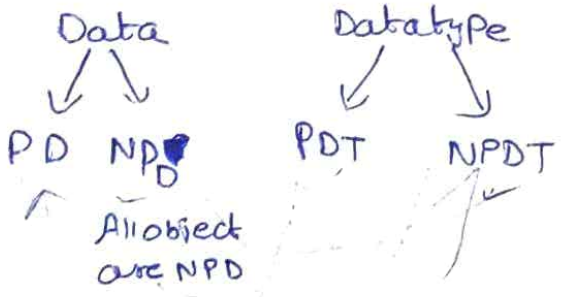
Day :- Thursday

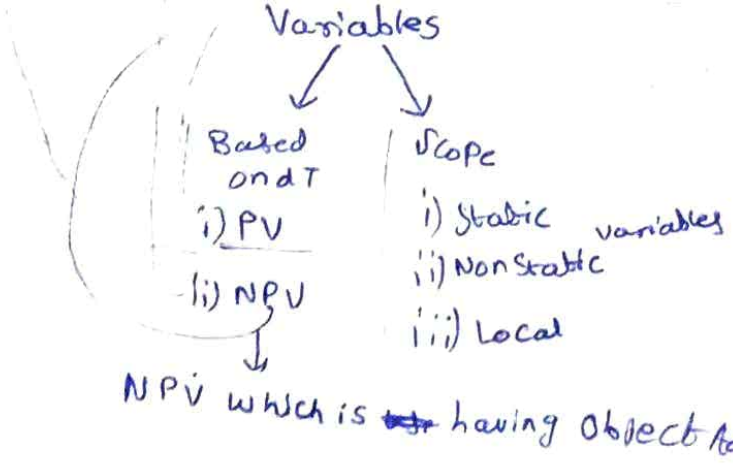
```
Class Person
{
// Properties [V]
// Behaviour [M]
}
```

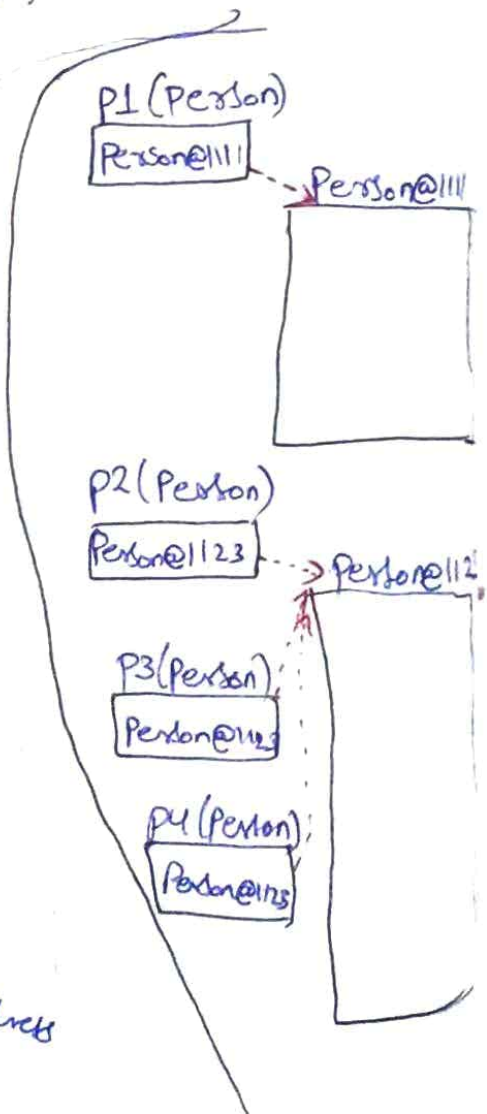
Method
:- AS M - RT methNF

```
Class PersonDriver
{
 Public Static Void main (String [ ] args) {
    → Person p1 = new Person( );
    → Person p2 = new Person( );
      Person p3 = p2;
      Person p4 = p3;
      System.out.println(p1==p2); //False
      System.out.println(p3==p4); // true
 }
}
```

Caller P.M Called
Method Method
return







Interview Questions

1. What is object and Why do we need object in Java
2. How to Create object
3. What is Class
4. Explain New Keyword
5. What is Object Reference Variable and How to create it.
6. what is driver class
7. Can One Object is having multiple object Reference Variable

Yes / NO

CHAPTER-2

Members of Class (Class Members):-

Any Members which is declared by using Class block is known as Members of class.

```
Class ClassName
{
    i) Variables
    ii) Methods
    iii) Constructor
    iv) Initializer.
}
```

(Class Block)

Day 53

Date :- 23/10/2026
Day :- Friday

Types of Class Members

ClassMembers

Static Members	Non Static Members
1) Static Variable	1) Non Static Variable
2) Static Method	2) Non Static Method
3) Static Initializer	3) Non Static Initializer
	4) Non Static Constructor

SP

*All the members can be declared only in Global Area

Static Members

→ Static Members can be declared by using static keyword.

→ Static is a keyword and Modifier.

→ When the Properties and behaviour of the object is Common for all the objects, then we should use Static Members.

i) Static Variable

Static Variable: Any Variable which is declared inside class block, by using static keyword is known as Static Variable.

```
Class Demo
{
 Static int a;
 Static double d;

}
```

All the Static Variables are initialized with default va

Datatypes	Default Value
Byte, Short, int, long	→ 0
Float, double	→ 0.0
Char	→ UniCode → Chardata ASCII Value is 0
boolean	→ false
All class Name [NPDT]	→ null.

ii) Static methods

Static methods :- Any methods which is declared by using static keyword is known as static methods.

Ex:-

```
class Demo
{
  Public static void m2( )
  {
    //SM
  }
  Public static void m2( )
  {
    //SM
  }
}
```

Accessability of Static Members.

We can Access static members in two different place

i) Same class

ii) Different class.

i) Same class

1. directly
2. ClassName [.]
3. Object Reference Variable [.]

```
//Same class
class Demo
{
  //SM
  main( )
  {
  }
}
```

```
// Same Class
Class Demo
{
 Static int a;
 Public Static Void view( )
 {
  System.out.println ("Demo Class SM");
 }
 Public Static Void main (String [ ]args)
 {
  // Directly
  System.out.println (a); // 0
  View( ); // Demo Class SM
  // ClassName[.] var Name.
  System.out.println (Demo.a); // 0
  Demo.View( ); // Demo Class SM
  //Object Reference Variable [.] var Name
  Demo d = new Demo( ); // 0
  System.out.println (d.a); // Demo Class SM
  d.view( );
 }
}
```

ii) Different Class.

1) Class Name [.]

2) Object Reference Variable [.]

```
//Different Class
Class Dolu
{
 Static double n;
 Public static void Show( ){
     S.o.pln ("Dolu Class SM")
}
```

```
//Different class
Class Bolu
{
  Public Static void main(String[ ]args)
  {
   // ClassName[.]
   S.o.pln(Dolu.n);
   Dolu.Show( );
  //Object Reference variable[.]
  Dolu d = new Dolu( );
  S.o.pln(d.n);// 0.0
  d.Show( );  // Dolu class SM
  }
}
```

```
//Different class
 Class p
 {
   //SM
 }
```

```
//Different class
 Class Q
 {
   main( )
   {

   }
 }
```

Day - 54

Date :- 24/10/2026
Day :- Saturday

Static Initializer

Static Initializer is used to initialize the static variable

Types

1 Single Line Static Initializer
2 Multi Line Static Initializer

1. Single Line Static initializer

Any variable which is declared inside the class block by using Static keyword and Initialize with the value in Same line is known as Single Line Static Initializer

```
class A
{
    Static int a = 50;
}
```

Static variable — Single Line Static Initializer.

2. Multi Line Static Initializer

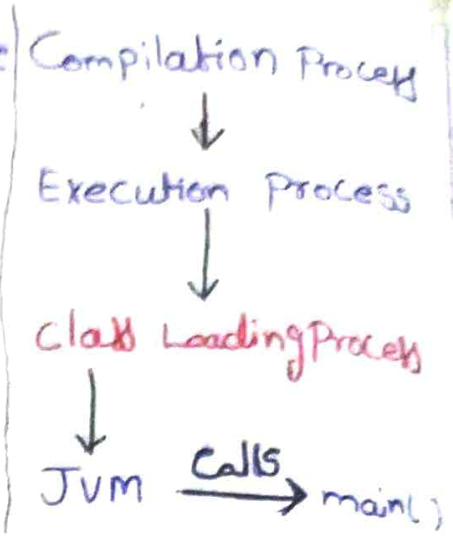
Any Block which is declared inside the class block by using Static keyword is known as Multi Line Static Initializer.

→ It is also known as Static block / Anonymous block.

```
class P
{
    static double n;
    Static
    {
      n = 5.56;
    }
}
```

→ In Java, All the Static initializers are automatically executed during class Loading process [Before Main method].

→ When class is having multiple static Initializer all the Initializers are executed in Sequential Order.




```
Class Test
{
    Static
    {
     System.out.println("Static Initializer - 1");
    }

    Static
    {
     System.out.println("Static Initializer - 2");
    }

    Public Static void main (String [] args) {
     System.out.println("Main Started");
     System.out.println("Main Ended");
    }
    Static
    {
     System.out.println("Static Initializer - 3");
    }
}
```

O/P:-

```
Static Initializer - 1
Static Initializer - 2
Static Initializer - 3
Main Started
Main Ended.
```

Day-55

Date: 26/12/2026
Day: Monday

Memory Allocation for static members

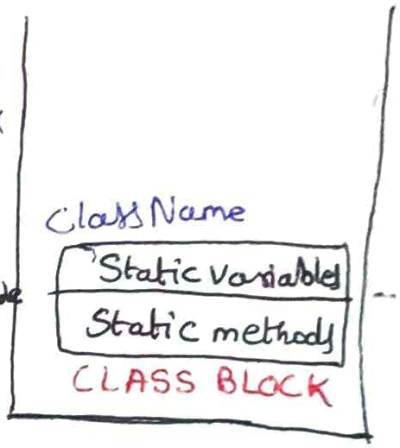
→ All the static members are allocating memory inside a class block which is present inside class static area

→ Every class is consist only 1 class block inside class static area

→ Always class block name is same name as ClassName

→ All static members are allocating the memory during class loading process in that time static initialization executed.

Class is a named block of memory which is used to store multiple static variable and multiple static methods. is known as CLASSBLOCK.



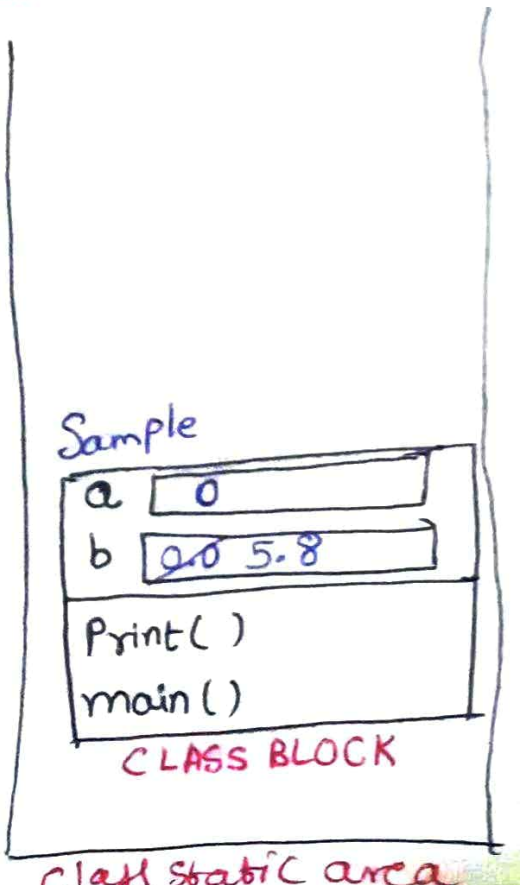
SA
HA

Class Static area.

i) Same class

```
Class Sample
{
  Static int a;
  Static int b = 5.8;
  public static void print()
  {
   //SM
  }
  public static void main()
  {

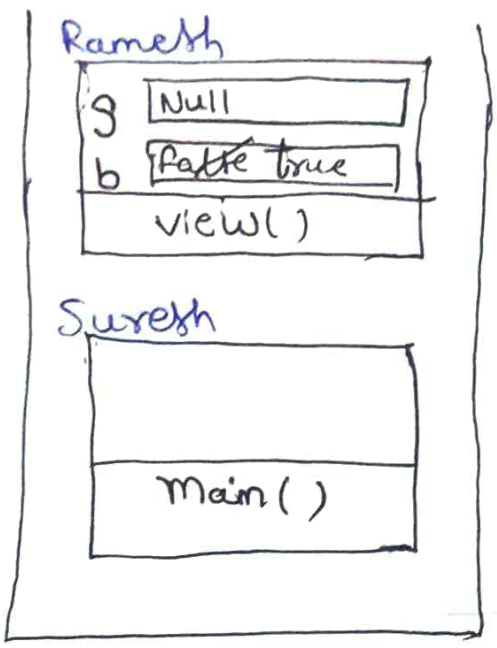
  }
}
```



ii) different classes

```
class Ramesh
{
Static String s;
Static boolean b = true;
Public Static Void view() {
  //SM
}
}
```

```
class Suresh
{
Public Static void main(String[] args)
{
System.out.println(Ramesh.s);
S.o.pln(Ramesh.b);
Ramesh.view();
}
}
```



※ Class Static area.

Conclusion for Static members

→ Static members are declared by using static keyword.

→ Static members are allocating the memory inside a class block which is present in class Static area.

→ All the Static initializers are executed automatically during class loading process.

→ When class is having multiple Static initializers it also executed automatically in Sequential Order (Top → bottom)

→ Object Creation is not mandatory, to access Static members because we can access by using ClassName

→ Static members are Single Copy.

→ When the properties and behaviour of object is common for all the objects, then we should use Static members.

Non Static members

→ Non Static members are declared Without using Static keyword.

→ When the Properties and, behaviour of the object is different for all the objects, then we should ute non Static members.

i) Non Static Variable

Any Variable which is declared inside class blocks without using Static keyword is known as Non Static Variable.

```
class A
{
    int p; //NSV
    double q; //NSV
    p.s.v.m(String[] args)
    {
    int a; //LV
    }
}
```

→ All Non Static Variables are also initialized with default values.

ii) Non Static Method

Any method which is not declared by using Static Keyword is known as Non Static Method.

```
Class R
{
    Public void view( )
    {
        //NSM
    }
    Void Print( )
    {
      //NSM
    }
}
```

Accessibility of Non Static members

→In Java, to access Non Static members, object Creation is mandatory then we use to access with the help of Object reference variable.

```
// Same class and different class using ObjRef Va
class Sample
{
    int a;
    Public void show( ){
    System.out.println ("Sample class NSM");
    }
    Public static void main (String [ ] args) {
    Sample s =new Sample ( );
                Obj; refers Variable
    System.out.println (s.a); // 0
    s.show( );   //Sample class NSM
    }
}
```

Day 56

Date: 27/10/2020
Day: Tuesday

Non Static Initializer: It is used to initialize the Non static Variable

i) Single Line Non Static Initializer
ii) Multi Line Non Static Initializer

i) Single Line Non Static Initializer

Any variable which is declared inside the class without using Static keyword and initializing the value in the same line is known as Single Line Non Static Initializer

```
class Demo
{
    int a = 100;
}
```

a → N.S.V
int a = 100; → Single Line Non Static Initializer.

ii) Multi Line Non Static Initializer

Any block which is declared inside the class without using any keyword is known as Multi Line Non Static Initializer.

→ It is also known as Non Static block/Anonymous block.

```
class Demo
{
    double a;
    {
    a = 9.4;
    }
}
```

→ In Java all the non static ~~keywords~~ Initializers, are automatically executed during Object Loading Process (during Object Creation)

→ Object Loading Process can also occurs for Multiple times When we can create multiple objects.

→ When the class is having multiple non static initializers it get executed in Sequential order.

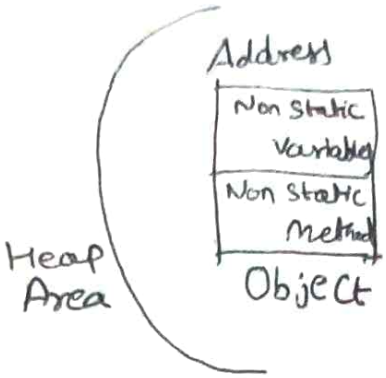
```
class Demo {
{
 S.o.pln ("Non Static Initializer-1");
}
{
 S.o.pln ("Non Static Initializer-2");
}
Public static void main(String[] args)
 {
 S.o.pln ("Main Started");
 Demo d1 = new Demo ();
 S.o.pln ("----------");
 Demo d2 = new Demo();
 S.o.pln ("Main Ended");
 }
 {
  S.o.pln ("Non Static Initializer-3");
 }
 }
```

O/P:- Main Started
Non Static Initializer - 1
Non Static Initializer - 2
Non Static Initializer - 3

Non Static Initializer - 1
Non Static Initializer - 2
Non Static Initializer - 3
Main Ended.

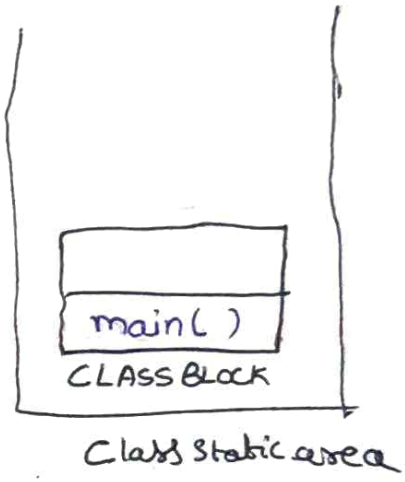
Memory Allocation for Non Static Members.

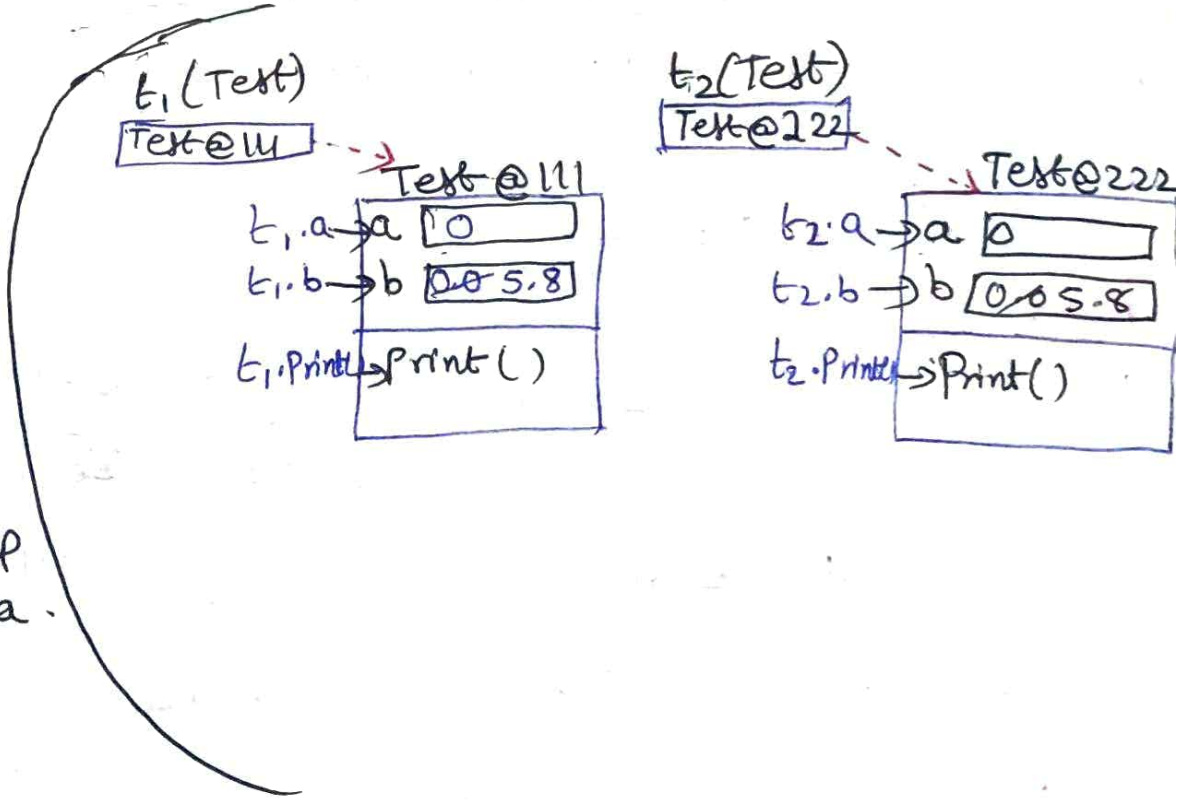
In Java all the Non Static members are allocating memory inside Object which is present in heap area.



Object is a address block of memory, which is used to store Multiple Non Static variables and Multiple Non Static Methods is known as Object.

```
Class Test
{
  int a;
  double b=5.8;
  Public void print()
  {
  //NSM
  }
  Public Static void main(String [ ]args)
  {
   Test t1 = new Test();
   Test t2 = new Test();
  }
}
```



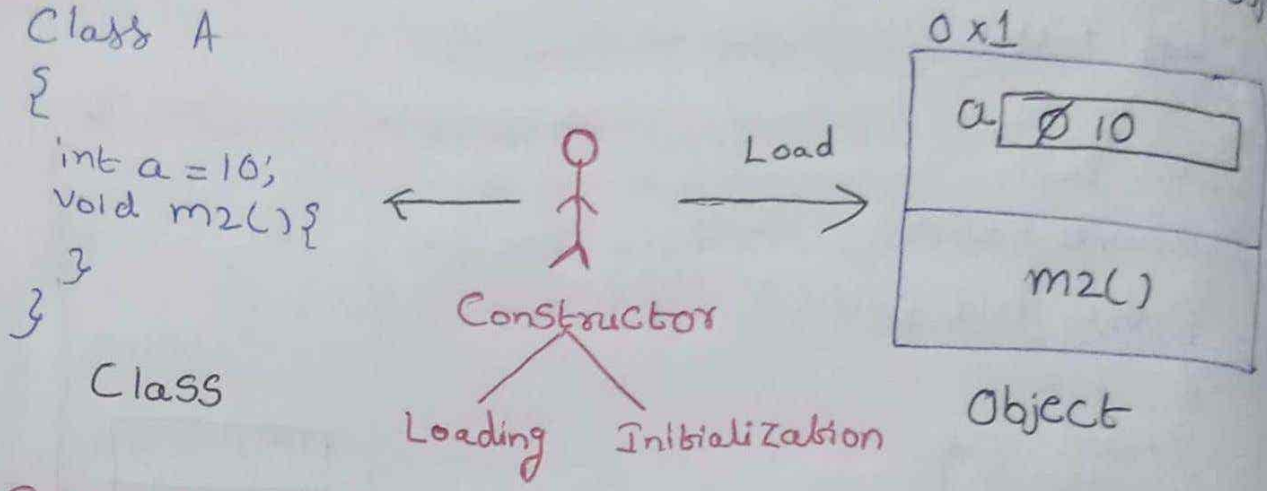


Day 57

Date:- 22/10/2025
Day:- Wednesday

Constructor

Constructor is a special Non static member, which is used to load all the non static members from class into object and it is also used to initialize the properties of the object.



Syntax to create a Constructor

Syntax:-

<AccessSpecifier> ConstructorName (<Formal Arguments>)
{
// Load Object Instruction [LOI]
Set of instructions for initialization
}

Constructor block

< > → Optional.

Rules for Constructor.

1. Constructor can be declared only inside Global Area
2. Constructor does not have Modifier and Return type.
3. Constructor Name must be same as class Name.

*

```
class Demo
{
  Demo(){
    //LOI
  }
}
```
✓

```
class Test
{
  Demo(){
  }
}
```
X

```
class Pen
{
  Static Pen(){
  }
}
```
✗

```
class Book
{
  Void Book(){
  } //NSI
}
```
✓

Note:

Constructor is executed only when Programmer is calling the constructor.

We can call the constructor only during object Creation.

```
new ClassName();
```

Keyword — Constructor calling statement

```
Class Sample {
    Sample() {
      //LOI
     System.out.println("Constructor");
    }

    Public static void main(String[] args) {
    System.out.println("Main Started");
    Sample S1 = new Sample();
    Sample S2 = new Sample();
    System.out.println("Main Ended");
    }
}
```

O/P:
```
main started
Constructor
Constructor
main ended
```

```
→main() {
→Sample S1 = new Sample();
  Sample S2 = new Sample();
  }
```

```
Sample() {
→//LOI
→S.o.pln("Constructor");
}
```

Default Constructor

When Programmer is not created any constructor, then Compiler is adding default Constructor during Compile time.

If Class is having Programmer defined constructor, then Compilier will not add default Constructor.

After Compilation, every class are having constructor.

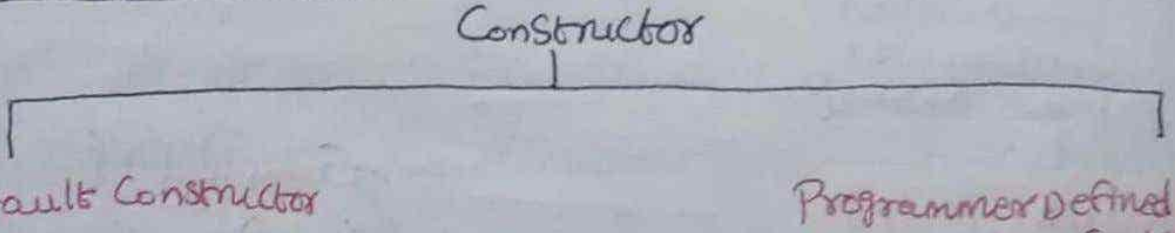
Before Compilation	After Compilation

Class Demo
{
 	
}
	Class Demo
	{
	Demo(){
	//LOI
	}
	}

Default Constructor

```
Class Demo {
    //Default Constructor
    Public Static Void main(String[]args){
    Demo d = new Demo(); //Calling default Constructor
    }
}
```

Types of Constructor



i) No Argument Constructor

ii) Parameterized Constructor

i) No Argument Constructor :-

Any Constructor which does not have Formal arguments is known as No Argument Constructor

ii) Parameterized Constructor

Any Constructor which is having Formal argument is known as Parameterized Constructor.

Day 58 Date: 29/01/2026
Day: Thursday

```
Class Test
{
   Test (int a, double b) {
   System.out.println ("a=" + a);
   System.out.println ("b =" +b);
   System.out.println ("------------");
   }
   Public static void main (String[] args) {
     Test t1 = new Test (120, 13.41);
     Test t2 = new Test (610, 123.74);
   }
}
```

O/P:- a = 120
b = 13.41
---
a = 610
b = 123.74

Class

Object Loading Process :-

When the Load object instructor is executed immediately Object loading Process started.

Step1:- Loading all the non static variable into the object and initialize with default value.

Step2:- Loading all the non static methods into the object

Step3:- Executing all non static Initializer from Top to bottom.

Once all the non static initializers are executed, then Object Loading Process will be Completed.

1. LOI
2. Why NSI 1st
3. Internal Procedure & object Creation.



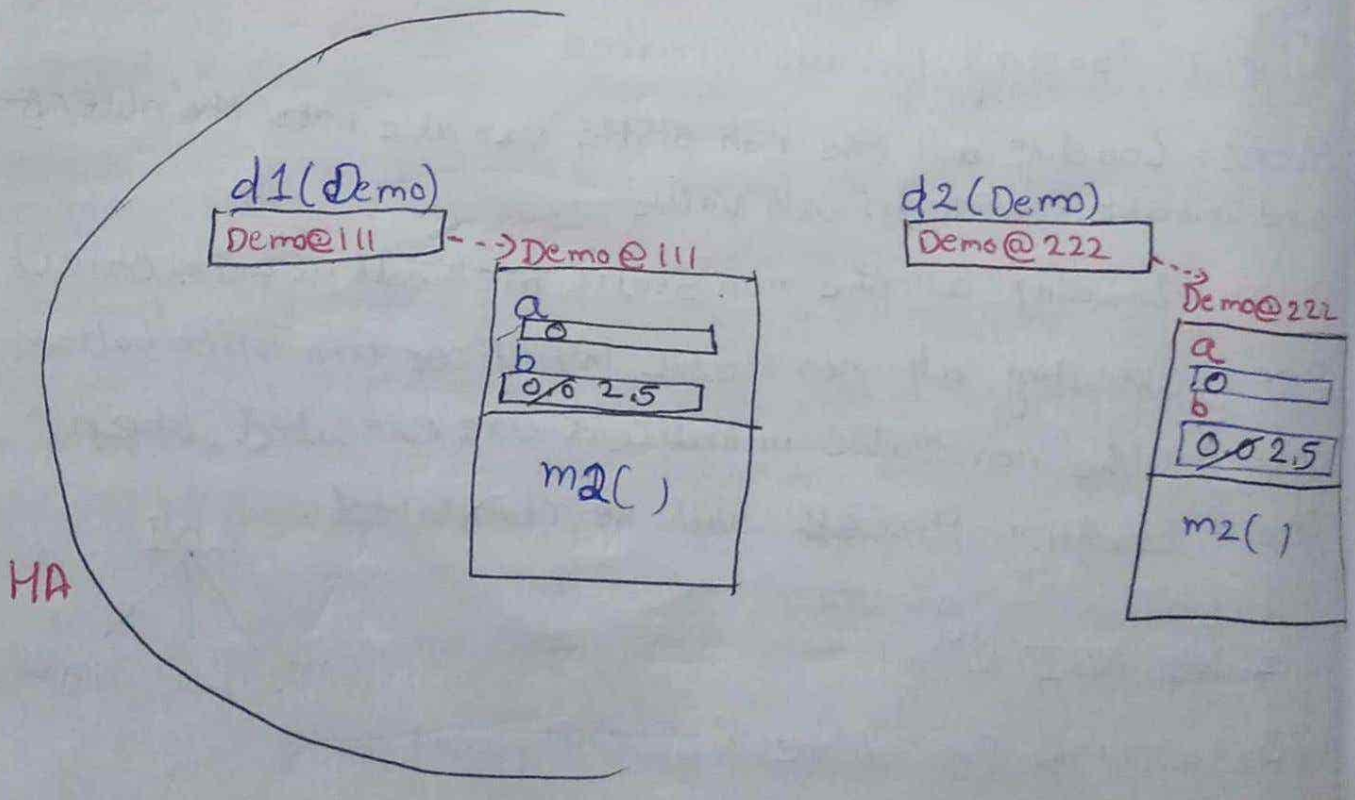
```
class Demo
{
  int a; → NSV
  double b = 2.5; → SL NSI
  Demo( )
  {                    //LOI
    System.out.println("Constructor");
  }

  Public void m2( ){
    System.out.println("NSM");
  }
  {
    System.out.println("ML-NSI");
  }
}
```

```
class DemoDriver
{
  P.S.V.main(String[ ]args)
  {
→ System.out.println("Main Started");
    Demo d1 = new Demo( );
    Demo d2 = new Demo( );
    System.out.println("Main Ended");
  }
}
```

O/p :- Main Started
ML-NSI
Constructor
ML-NSI
Constructor

main( ) → Demo( )



Day-59

Date: 30/10/2026
Day: Friday

Conclusion for Non Static members

→ Non Static members are declared without using Static keyword.

→ Non static members are allocating memory inside object, which is present in heap area.

→ All the non Static initializers are automatically executed during object loading process

→ When the class is having multiple Non Static initializers it is executed in Sequential Order.

→ Object Creation is mandatory to access Non Static member.

→ Non Static members are multiple copies.

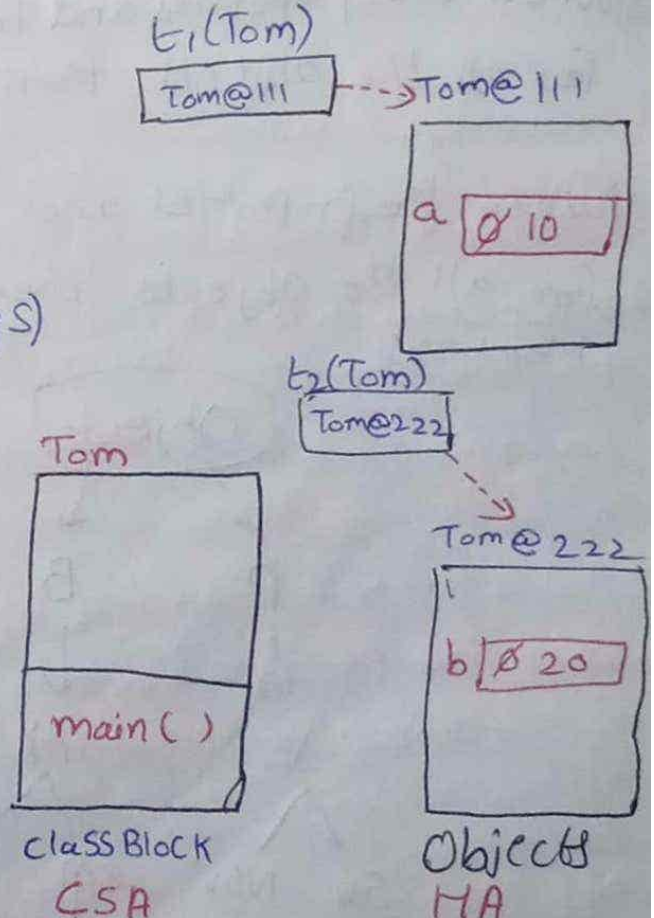
→ When Properties and behaviour of object is different for all the objects then we should use non Static members

Note:

1) When we modify the non static members in one object, it is not affected for another object because non static members are multiple copy.

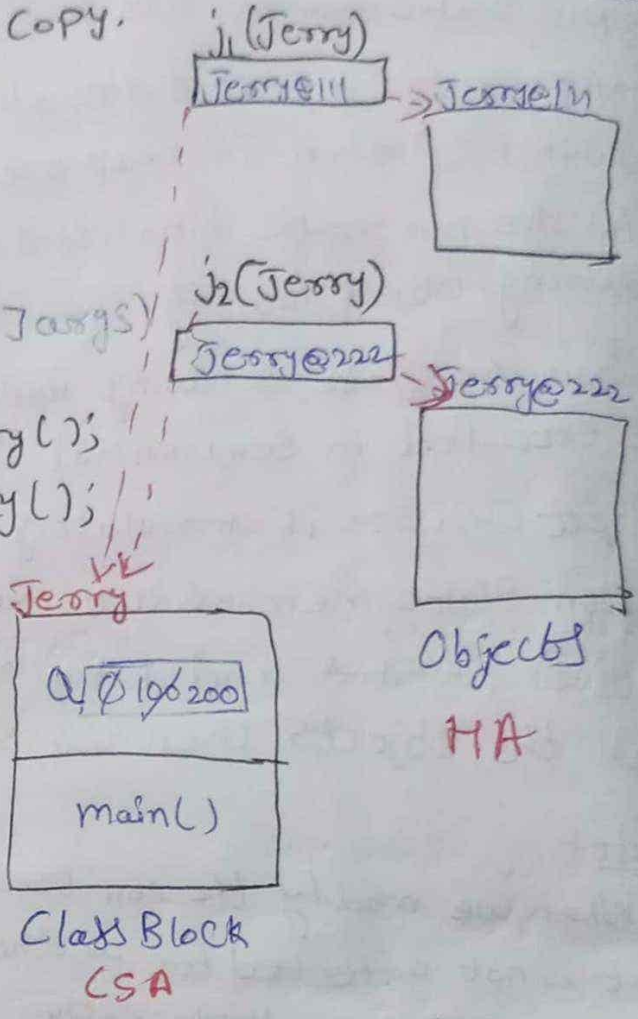
```
class Tom
{
    int a;

    P.S.V main(String[] args)
    {
        Tom t1 = new Tom();
        Tom t2 = new Tom();
        t1.a = 10;
        t2.a = 20;
        S.O.Pln(t1.a);
        S.O.Pln(t2.a);
    }
}
```



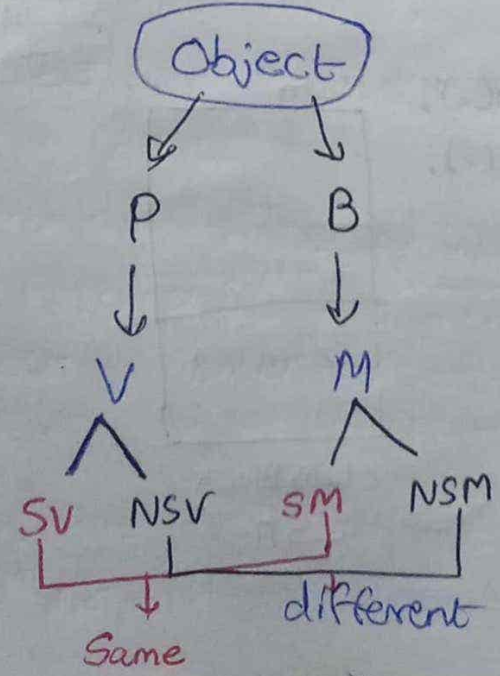
2) When we modify the data in static members inside a Class block, it will be affected for all the objects because Static members are single copy.

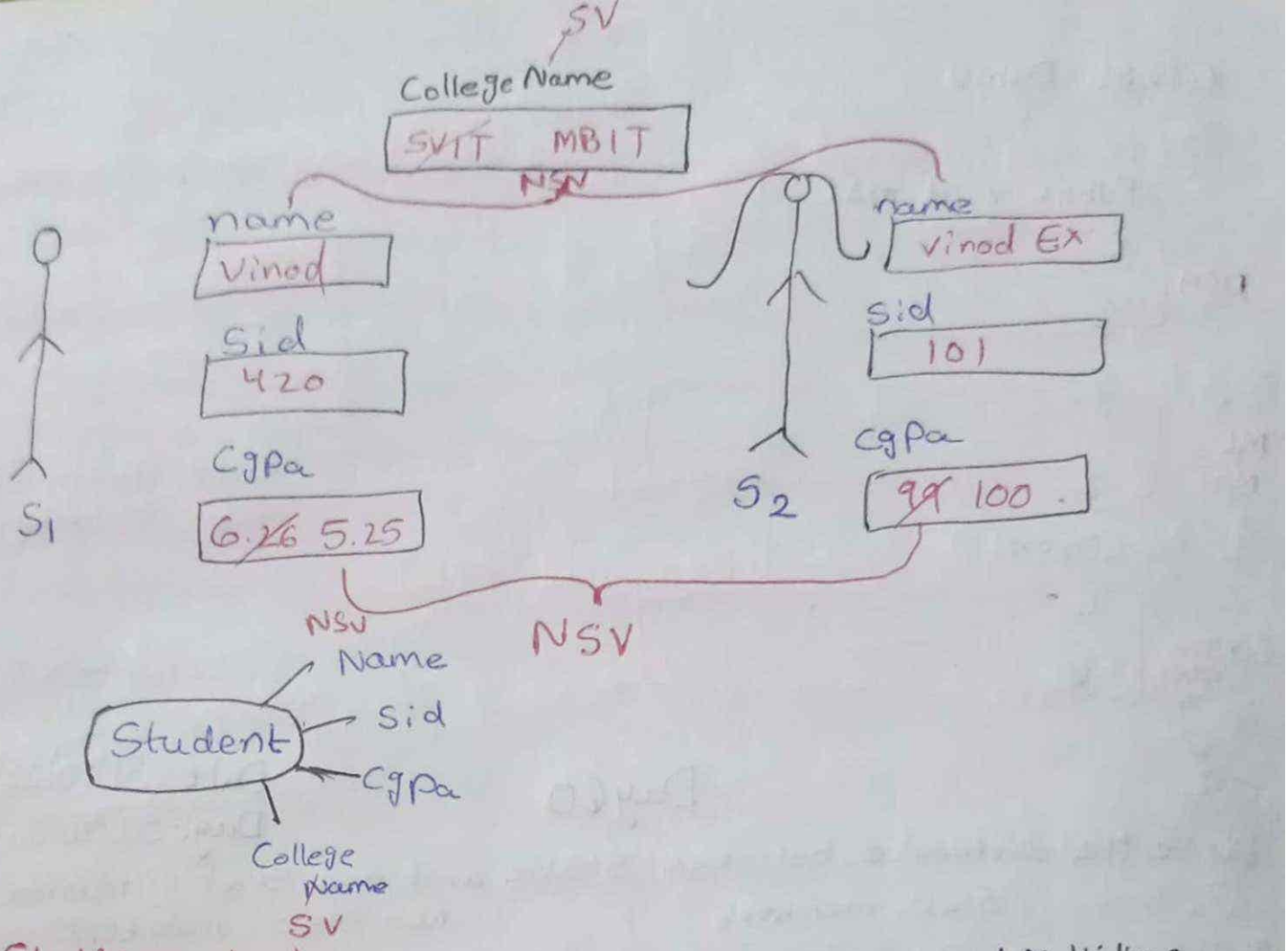
```
Class Jerry
{
    Static int a;
    P.S.V.main(String[] Jargs)
    {
      Jerry j1 = new Jerry();
      Jerry j2 = new Jerry();
      j1.a = 100;
      j2.a = 200
      S.o.pln(j1.a);
    } S.o.pln(j2.a);
}
```



3) When the Properties and behaviour of object is same for all the objects then we should use Static members

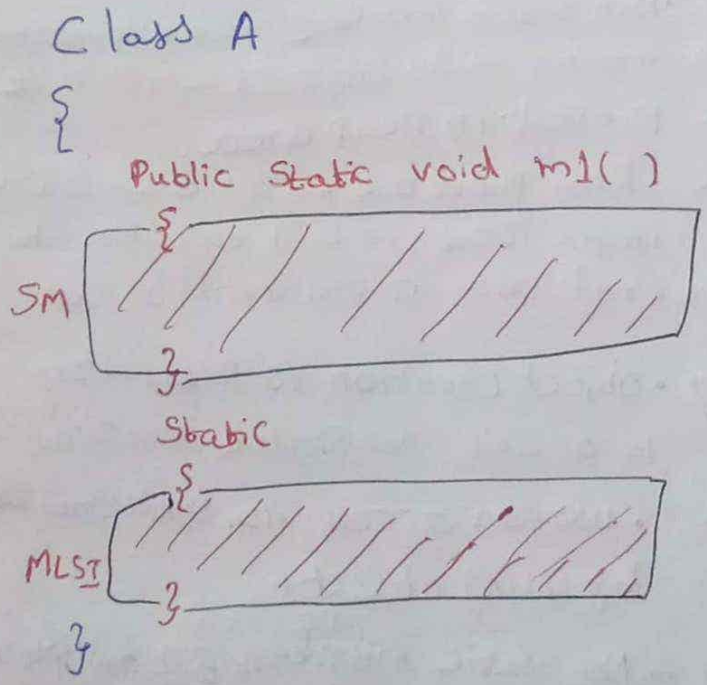
4) When the properties and behaviour of object is different for all the objects then we should use non Static members

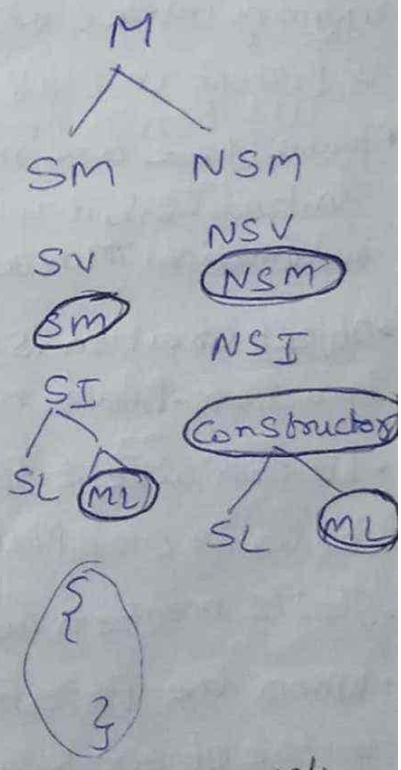




Static Context

The area which is belong to static method and Multi line static Initializer is known as Static context.





Non static Context.

The area which is belong to non static method, Multiline Non static Initializer and Constructor is known as Non static Context.

Class Demo
{
Public void m2( )
{
}
NSM

{
}
ML
NSI

Demo( )
{
}
Constru
ctor
}

Day 60

Date:- 31/10/2026
Day:- Saturday

Write the difference between Static and non Static members

Static members	Non Static members
• Static members are declared by using Static keyword	• Non Static members are declared by using without Static Keyword
• Static members are allocated memory inside class block which is present in class Static area	• Non Static members are allocated memory inside objects which are present in Heap area.
• When there are multiple static initializers, it will execute automatically in Sequential order	• When there are multiple non static initializers, it will executed automatically in sequential order.
• Object creation is not mandatory to access Static members	• Object creation is Mandatory to access non Static members.
• We can access Static members by using ClassName	• We can access non Static members by using objects.
• Static members are single Copy	• Non Static members are multiple Copy
• When the Properties & Behaviour of the object is common for all the objects then we use Static members	• When the Properties and behaviour of the object is different for all the objects then we use non Static members.
• All Static members allocating memory during Class loading Process	• All non Static members allocating memory during object loading Process

- Static members are having only 1 class block.
- Static members are divided into 3 types
- When we modify the data in static member. inside a class block. it will effects for all the objects.
- Static members does not have Load Object instructor.
- Static members will execute before main method.

- Non Static members, we can create multiple objects
- Non Static members are divided into 4 types
- When we modify the non static member in one object it is not effect for other object.
- Non static members has Load object instructor.
- Non Static members will execute during Creation of object.

Accessability of static and non static members inside static and non static Context

How to access static member inside Non static method during Same class?

Members	Static Context [Same class]	Static Context [Different class]	Non Static Context [Same class]	Non Static Context [Different class]
Static Member	* 1 Directly 2 className 3 Object ref Variable	* 1 ClassName 2 Object ref Variable	* 1 Directly 2 className 3. object ref Variable	* 1 ClassName 2 Object ref variable
Non-Static Member.	1 Object ref Variable	1 Object ref Variable	* 1 Directly 2 Object ref Variable	1 Object ref Variable

How to access Non static member inside Non Static Initializer during Same class?

Static Context	Non Static Context
1. SM	1. NSM
2. ML SI	2. ML NSI
	3. Constructor.

CHAPTER 8

Initializing Properties of Object

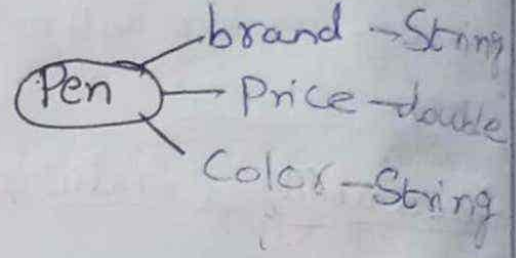
There are 3 ways to initialize the Properties of Object

1. Non Static Initializer
2. Object Reference variable
3. Parameterized Constructor

Day-61

Date: 02/11/2026
Day: Monday.

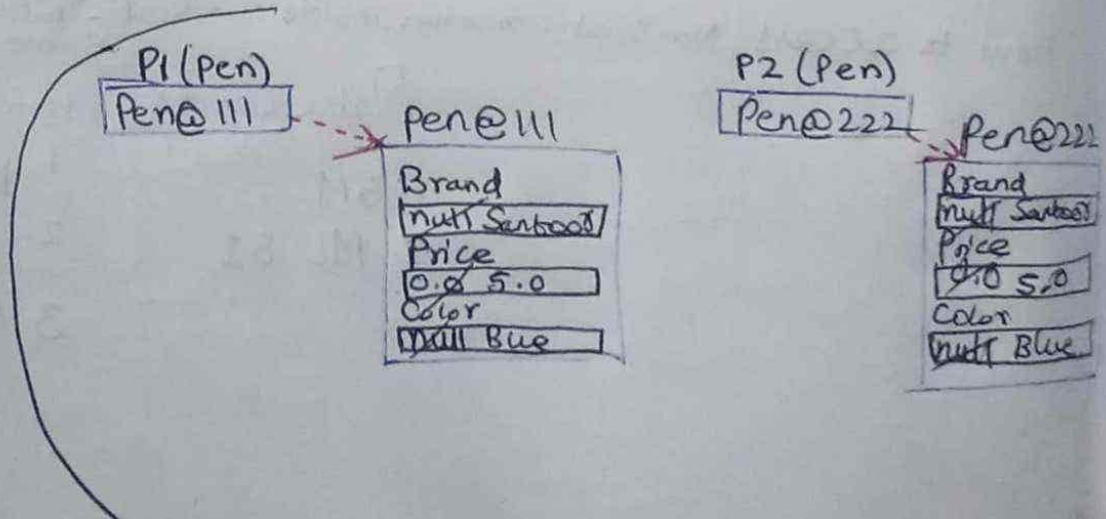
1. Non Static Initializer [SL / ML]




```
Class Pen
{
  String brand = "Santoor";
  double Price = 5.0;
  String color = "Blue";
  Pen () {
    //LOI
  }
}
```

NSV ✓
NSM
NSI ✓

```
Class PenDriver
{
  P.S.V.main (String[]args)
  {
    Pen P1 = new Pen();
    Pen P2 = new pen();
  }
}
```



H.A

Note:- Every objects are initialized by same value when we use Non Static Initializer.

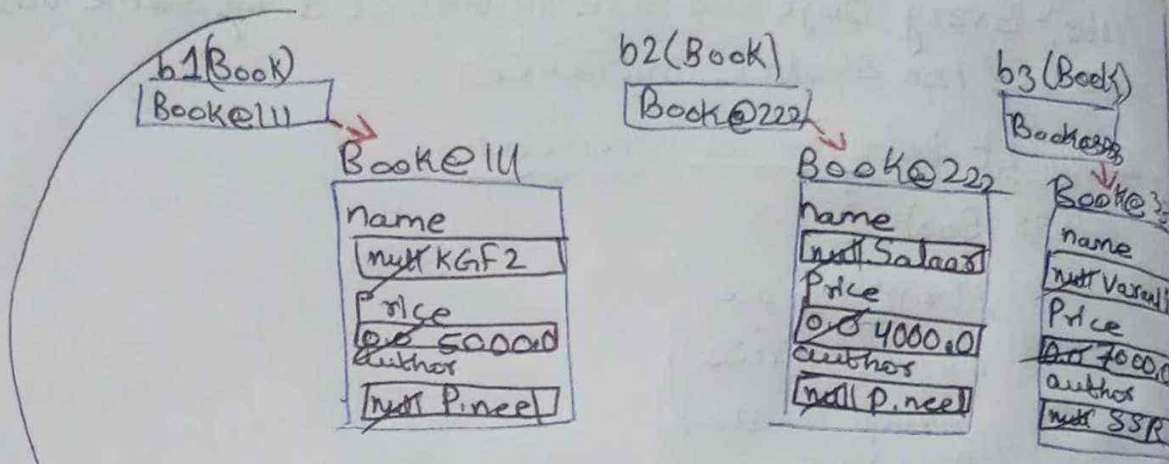
2) Object Reference Variable.

```
Class Book {
    String name;
    double Price;
    String author;
    Book() {
    //LOI
    }
    Public void PrintBookDetails() {
    S.o.pln ("Name:" +name);
    S.o.pln ("Price:" + price + "RS");
    S.o.pln ("Author:" +author);
    S.o.pln ("----------");
    }
}
```

NSV ✓
NSM ✓
NSI ✓

```
Class BookDriver {
    Public Static Void main (String[] args) {
    Book b1 = new Book();
    b1.name = "KGF-2";
    b1.price = 5000.0;
    b1.author = "Pineel";
    b1.PrintBookDetails();
    Book b2 = new Book();
    b2.name = "Salaar";
    b2.price = 4000.0;
    b2.author = "Pineel";
    b2.PrintBookDetails();
    Book b3 = new Book();
    b3.name = "Varanasi";
    b3.price = 7000.0;
    b3.Author = "SSR";
    b3.PrintBookDetails();

    }
}
```



H.A.

Note:- Length of code is increasing when we initialize the Properties of the object by using object Reference Variable.

Day 62

Date:- 03/11/202
Day:- Tuesday

3. Parameterized Constructor